UNIVERSIDAD DA VINCI

DE GUATEMALA

ESTRUCTURA DE DATOS

Primer Parcial

PRESENTA

LUIS COJOM

CARNE: 202503631

GUATEMALA 03/03/2026

Introducción

Como desarrolladores sabemos que es importante conocer que tan eficientes son nuestros programas, por ejemplo, cuanto tiempo se tarda nuestro programa en procesar n documentos, n numero de transacciones, que tanta memoria ocupa.

Es por ello que existe la métrica teórica en la informática llamada Big O. Lo que hace esta métrica es describir la eficiencia y escalabilidad de un algoritmo, con el fin de clasificarlo según cuanto tiempo o espacio de memoria requiere en relación al tamaño de la entrada. Normalmente al realizar esta medición se suele enfocar en el peor de los casos.

Descripción de algoritmos

Algoritmo Factorial:

Este algoritmo calcula el producto de todos los números desde 1 hasta n. Es decir, si se solicita calcular la factorial de 5, la operatividad es: (1*2*3*4*5) = 120

Algoritmo Fibonacci:

Este algoritmo devuelve el resultado n que es producto de la suma de los números anteriores, comenzando con sus dos valores por default que son 0 y 1. Es decir, si se solicita obtener el valor Fibonacci 3, la operatividad es: (0+1+1) = 2

Búsqueda Lineal:

Este algoritmo explora de forma secuencial cada elemento del arreglo hasta localizar el valor objetivo y devolver la posición en la que se encuentra dentro del arreglo.

Algoritmo Burbuja:

Este es un algoritmo de ordenamiento por medio del intercambio de posiciones de los valores que están dentro de un arreglo, la manera en que lo hace es comparando elementos y validando que estén en el orden correcto ascendente.

Fragmentos de código

Algoritmo Factorial Iterativo:

```
long resultado = 1;
if (n == 0 || n == 1) return resultado;
for (int i = 1; i <= n; i++) {
    resultado = resultado * i;
}
return resultado;
```

Algoritmo Factorial Recursivo:

```
if (n == 0 || n == 1) return 1;
else {
    return factorialRecursivo(n - 1) * n;
}
```

Algoritmo Fibonacci Iterativo:

```
if (numeroLimite <= 1) return 1;

long a = 0, b = 1, suma;
for (int i = 2; i <= numeroLimite; i++) {
    suma = a + b;
    a = b;
    b = suma;
}
return b;
```

Algoritmo Fibonacci Recursivo:

```
if  (numeroLimite <= 1) return numeroLimite;
return fibonacciRecursivo(numeroLimite - 1) + fibonacciRecursivo(numeroLimite - 2) ;
```

Algoritmo Búsqueda Lineal Iterativo:

```
for(int i = 0; i < arreglo.length; i++){
    if (arreglo[i] == numeroAEncontrar){
        return i + 1;
    }
}
return -1;
```

Algoritmo Búsqueda Lineal Recursivo:

```
if (indice >= arreglo.length)
    return -1;
else if (arreglo[indice] == objetivo)
    return indice + 1;

return busquedaLinealRecursiva(arreglo, objetivo, indice - 1);
```

Algoritmo Burbuja Iterativo:

```
while (marcaOptimizacion > 0) {
    int nuevaMarca = 0;

    for (int i = 0; i < marcaOptimizacion; i++) {
        int vIzquierda = arreglo[i];
        int vDerecha = arreglo[i + 1];

        if (vIzquierda > vDerecha) {
            arreglo[i] = vDerecha;
            arreglo[i + 1] = vIzquierda;
            nuevaMarca = i;
        }
    }
    marcaOptimizacion = nuevaMarca;
}
return arreglo;
```

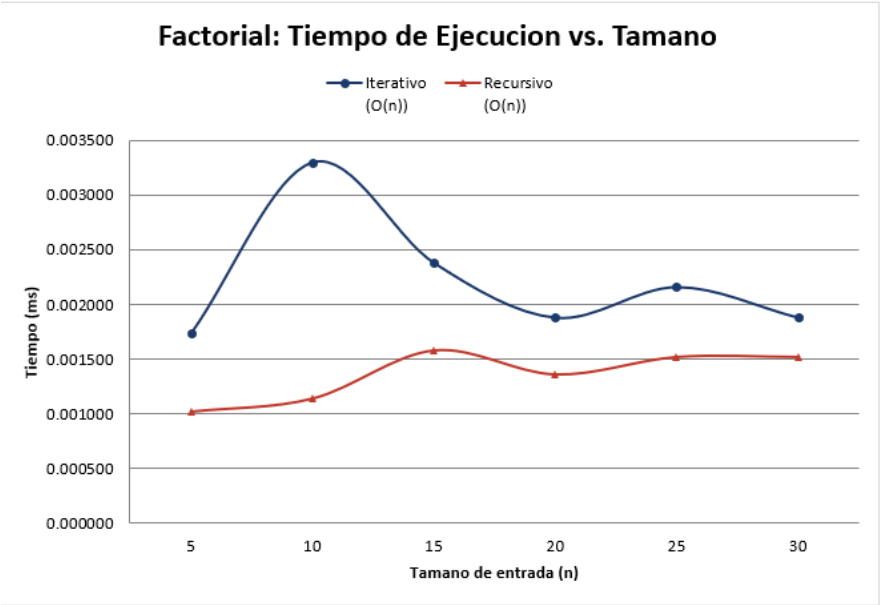
Algoritmo Burbuja Recursivo:

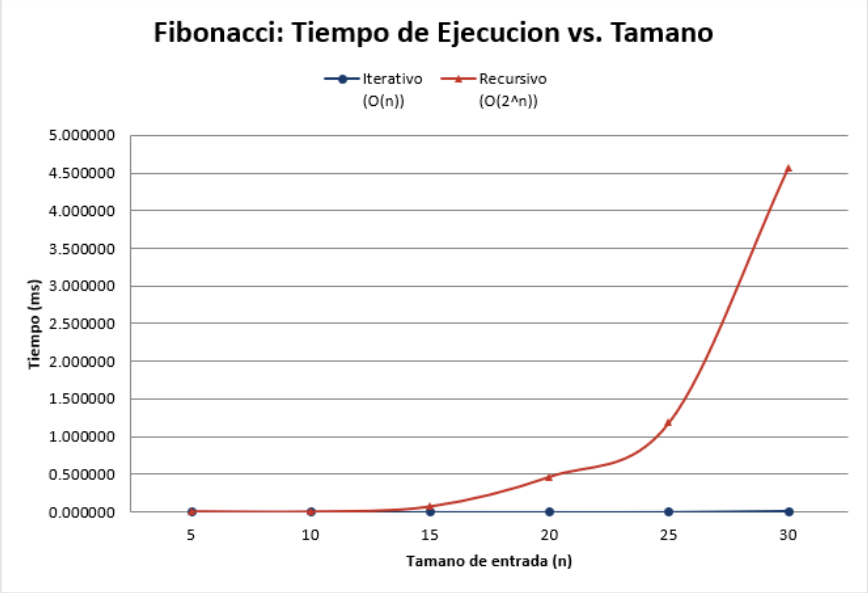
```
for (int i = 0; i <= longitudArreglo - 1; i++){
    if (arreglo[i] > arreglo[i+1]){
        int temp = arreglo[i];
        arreglo[i] = arreglo[i+1];
        arreglo[i+1] = temp;
    }
}
burbujaRecursivo(arreglo,longitudArreglo -1);
```

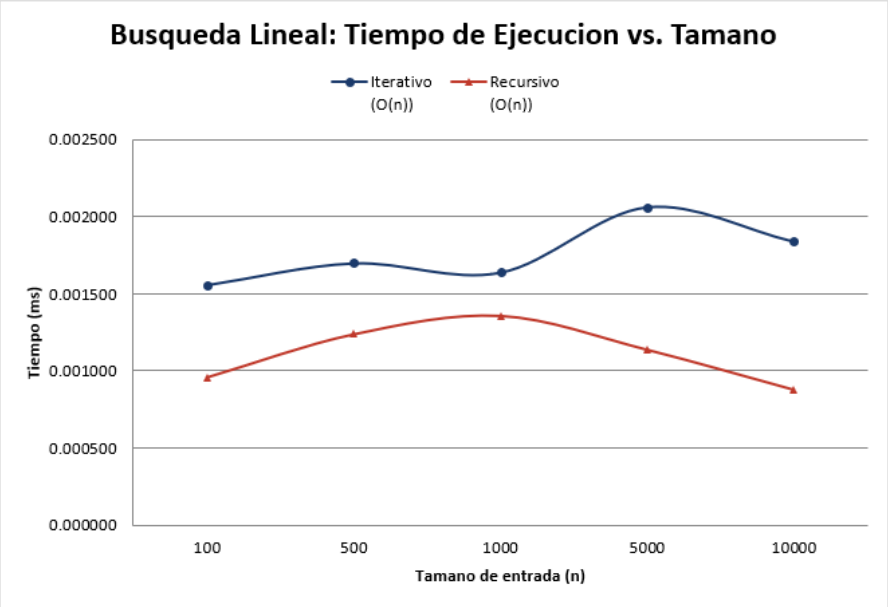
Tabla de tiempos medidos

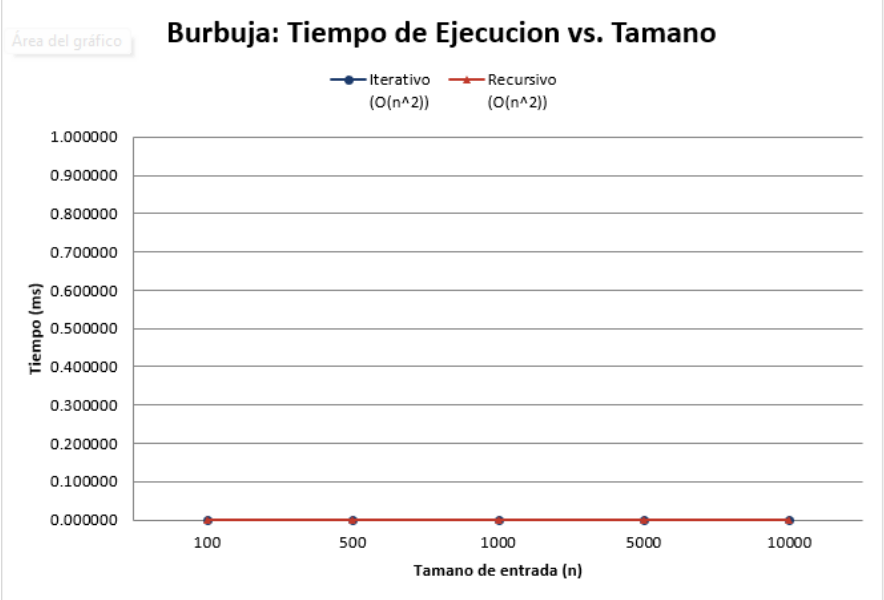
Algoritmo	Version	n	Intentos consecutivos	Resultado	Tiempo_ms
Fibonacci	Iterativo	5	5	5	0.00058
Fibonacci	Iterativo	10	5	55	0.0006
Fibonacci	Iterativo	15	5	610	0.00092
Fibonacci	Iterativo	20	5	6765	0.00076
Fibonacci	Iterativo	25	5	75025	0.00076
Fibonacci	Iterativo	30	5	832040	0.00088
Fibonacci	Recursivo	5	5	5	0.0007
Fibonacci	Recursivo	10	5	55	0.01048
Fibonacci	Recursivo	15	5	610	0.06306
Fibonacci	Recursivo	20	5	6765	0.11604
Fibonacci	Recursivo	25	5	75025	0.50796
Fibonacci	Recursivo	30	5	832040	4.09004
Factorial	Iterativo	5	5	120	0.0005
Factorial	Iterativo	10	5	3628800	0.0006
Factorial	Iterativo	15	5	1307674368000	0.0006
Factorial	Iterativo	20	5	2432902008176640000	0.00072
Factorial	Iterativo	25	5	7034535277573960000	0.00072
Factorial	Iterativo	30	5	-8764578968847250000	0.00082
Factorial	Recursivo	5	5	120	0.0004
Factorial	Recursivo	10	5	3628800	0.0005
Factorial	Recursivo	15	5	1307674368000	0.0006
Factorial	Recursivo	20	5	2432902008176640000	0.00082
Factorial	Recursivo	25	5	7034535277573960000	0.00096
Factorial	Recursivo	30	5	-8764578968847250000	0.00142
Busqueda Lineal	Iterativo	{1000,500,100,5000,10000}	5	5	0.00056
Busqueda Lineal	Recursivo	{1000,500,100,5000,10000}	5	5	0.00026
Burbuja	Iterativo	{1000,500,100,5000,10000}	5	[100, 500, 1000, 5000, 10000]	0.00068
Burbuja	Recursivo	{1000,500,100,5000,10000}	5	[100, 500, 1000, 5000, 10000]	0.0006

Graficas


Factorial: Tiempo de Ejecucion vs. Tamano
Iterativo (O(n))
Recursivo (O(n))
Tiempo (ms)
0.003500
0.003000
0.002500
0.002000
0.001500
0.001000
0.000500
0.000000
5
10
15
20
25
30
Tamano de entrada (n)


Fibonacci: Tiempo de Ejecucion vs. Tamano
Iterativo (O(n))
Recursivo (O(2^n))
Tiempo (ms)
5.000000
4.500000
4.000000
3.500000
3.000000
2.500000
2.000000
1.500000
1.000000
0.500000
0.000000
5
10
15
20
25
30
Tamano de entrada (n)


Busqueda Lineal: Tiempo de Ejecucion vs. Tamano
Iterativo (O(n))
Recursivo (O(n))
0.002500
0.002000
0.001500
0.001000
0.000500
0.000000
Tiempo (ms)
100
500
1000
5000
10000
Tamano de entrada (n)


Burbuja: Tiempo de Ejecucion vs. Tamano
Iterativo (O(n^2))
Recursivo (O(n^2))
1.000000
0.900000
0.800000
0.700000
0.600000
0.500000
0.400000
0.300000
0.200000
0.100000
0.000000
Tiempo (ms)
100
500
1000
5000
10000
Tamano de entrada (n)

Análisis y conclusiones

La versión recursiva es drásticamente más lenta porque su complejidad es O(2^n). Cada llamada genera un crecimiento exponencial de funciones en la memoria, provocando que el tiempo de ejecución se dispare incluso con valores pequeños de n.

O(n^2) Indica una complejidad cuadrática; esto significa que el tiempo de ejecución es proporcional al cuadrado del número de elementos.

El uso de recursión en Java implica un alto consumo de la pila de llamadas (stack), ya que cada invocación almacena un nuevo marco de activación. Esto puede derivar en un **StackOverflowError** si la profundidad de la recursión excede los límites del sistema.

Se recomienda la recursividad para problemas con estructuras fractales o jerárquicas (como árboles) donde el código resulta más mantenible. Se prefiere la iteración cuando el rendimiento y la eficiencia de memoria son la prioridad absoluta.